CRISIS Landlab Component



Coupled Regional Rainfall-Induced and Seismic Slope Instability Simulations

Authors: Mirna Kassem & Dimitrios Zekkos
Affiliation: University of California, Berkeley
Contact: kassem_mirna@berkeley.edu · zekkos@berkeley.edu

About

CRISIS (Coupled Regional Rainfall-Induced and Seismic Slope Instability Simulations) is an open-source, Python-based model for simulating **shallow** rainfall- and earthquake-triggered landslides at regional scales. CRISIS evaluates slope stability across a raster domain and generates realistic pseudo-3D landslide geometries ("scoops"), each characterized by its location, area, volume, failure surface, and timing of failure (when coupled with a transient groundwater flow model).

CRISIS is also available as a standalone code on DesignSafe, where it has two modes: forward prediction of landslides, and back-analysis of mapped landslides to estimate shear strength parameters. Only the forward prediction mode is included in Landlab.

This manual documents both parts of CRISIS for Landlab: the **CRISIS core** (`_crisis_core.py`), which contains the physics, and the **CRISIS Landlab component** (`crisis_landlab_component.py`), through which CRISIS is run. Section 3 explains what each part does; the rest of the manual describes what the user provides, every parameter, the outputs, how to run CRISIS (with a complete example), and the model physics.

The model is designed to be highly flexible, accommodating a wide range of user needs, from simple simulations using uniform shear strength and a constant pore pressure coefficient, to advanced configurations with spatially variable 3D strength fields, transient pore pressure heads, and coupled seismic displacement models.

CRISIS accepts pore pressure heads that vary in space and time, computed by an external transient groundwater flow model (as HDF5 files) or by a coupled Landlab component, enabling time-evolving stability assessments throughout a storm sequence. As a simple alternative, a steady-state, spatially uniform pore pressure coefficient R_u can be used. For coupled rainfall-earthquake scenarios, seismic displacement is estimated using established empirical models.

Table of Contents

1. Citing CRISIS
2. Overview
3. Code Structure: Core and Component
4. Installation
5. Inputs
6. Pore Pressures and Water Contents
7. Parameters (the `params` Dictionary)
8. Outputs
9. Running CRISIS in Landlab
10. Model Physics

---

1. Citing CRISIS

If you use CRISIS in published research, please cite:

Kassem, M., & Zekkos, D. (2026). Assessing rainfall-induced landslide failure mechanisms in regional hydrological and hillslope stability simulations. *Landslides*, 1–20. https://doi.org/10.1007/s10346-025-02690-w

---

2. Overview

CRISIS is a spatially distributed, Python-based model for regional hazard assessment of shallow landslides. It evaluates slope stability across a raster domain and reconstructs realistic failure geometries using a pseudo-3D approach.

Two triggering scenarios are supported:

- **Rainfall only:** failures driven by pore water pressure
- **Coupled rainfall + earthquake:** failures driven by pore water pressure and seismic displacement combined

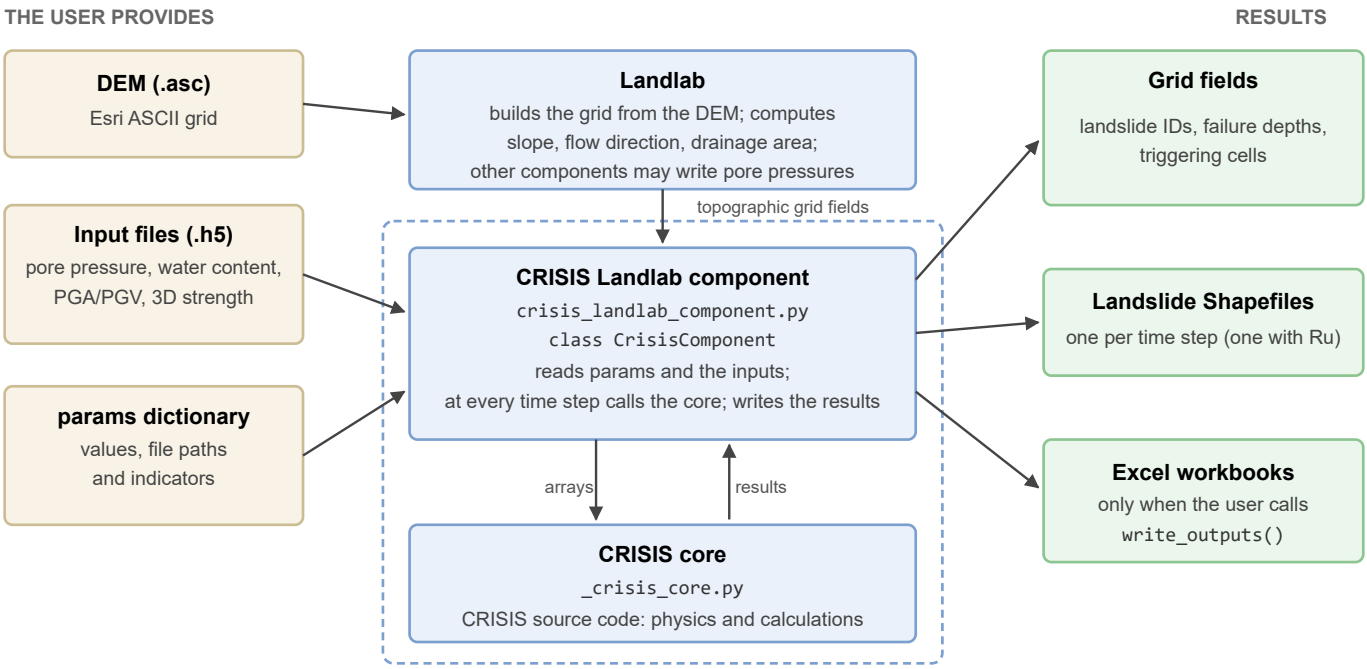
The model predicts landslide occurrence, area, volume, spatial extent, and time of failure (if transient pore pressure heads are provided) across a raster domain for a given set of shear strength parameters.

---

3. Code Structure: Core and Component

CRISIS for Landlab consists of two Python files that work together: the **CRISIS core**, which is the CRISIS source code, and the **CRISIS Landlab component**, which runs the core within Landlab.

File	Name used in this manual	What it does
`_crisis_core.py`	**CRISIS core**	The CRISIS source code: the physics and calculations of the model. The user does not call it directly; the component calls it.
`crisis_landlab_component.py`	**CRISIS Landlab component**	The part the user works with. It defines the Python class `CrisisComponent`, which the user creates with the Landlab grid and the `params` dictionary (Section 7): `CrisisComponent(grid, params)`. It reads the inputs, calls the core at every time step, and writes the results to Landlab grid fields and shapefiles. The Excel workbooks are written only when the user calls `write_outputs()` (Section 9).



How the two parts work together. The user provides the DEM, the input files, and the `params` dictionary. Landlab builds the grid from the DEM and computes the topographic grid fields. The component reads `params` and the inputs; at every time step it passes arrays to the core, receives the results, and writes them to grid fields and shapefiles.

In practice, the user never calls the core directly: everything is done through the component. The rest of this manual is organized accordingly:

- **Section 5, Inputs:** the files the user provides, their format, and how raster files must be stored.
- **Section 6, Pore Pressures and Water Contents:** the three ways of supplying them, and every combination.
- **Section 7, Parameters:** every key of the `params` dictionary.
- **Section 8, Outputs:** the grid fields and the output files.
- **Section 9, Running CRISIS in Landlab:** the steps of a run, a complete example, the component's methods, and the example notebook.
- **Section 10, Model Physics:** the equations implemented in the core.

---

4. Installation

Requirements

- Python 3.10 or newer (required by Landlab 2.x)

Install all required packages in one step with the `requirements.txt` file provided alongside the code:

```
pip install -r requirements.txt
```

To run the example notebook (`Example/CRISIS_Example.ipynb`, Section 9), also install Jupyter and the interactive widgets:

```
pip install jupyter ipywidgets
```

Dependencies

Package	Purpose
`numpy`	Array math and raster operations
`h5py`	Reading HDF5 files (pore pressure heads, water content, PGA/PGV, 3D strength arrays)
`geopandas`, `shapely`, `pyproj`	Landslide polygon export (Shapefile)
`pandas`, `openpyxl`	Results export (Excel)
`landlab`	Grid framework of the CRISIS Landlab component; also used to read the ASCII DEM and to compute slope, flow direction, and drainage area

---

5. Inputs

The user provides all inputs in the Python script that calls and runs the CRISIS Landlab component (Section 9). There are two kinds of input:

1. **Files:** the DEM and, depending on the settings chosen, raster files of pore pressure, water content, ground motion, and soil strength. They are listed in the table below, with their format.
2. **Parameters:** the model settings (soil properties, depths, indicators, …) and the locations of the files, given as the keys of the `params` dictionary (Section 7).

The table lists every file the user can provide. Only the DEM is always required; the other files are needed only for the settings shown. The column *Parameter* gives the key of the `params` dictionary that holds the location of each file. Slope, flow direction, and drainage area are not inputs: they are computed by Landlab from the DEM before the CRISIS Landlab component is created (Section 9).

Input	Parameter (key in `params`)	File format	Required when
DEM, ground elevation (m); its header also gives the grid size, cell size, and coordinates	`dem_file`	Esri ASCII grid (`.asc`)	Always (Section 5, DEM)
Pore pressure head ψ (m)	`hydro_files_dir`	HDF5 (`.h5`), one 2D file per time step and depth layer	`Hydrological_model_indicator = 2` (Section 6, HDF5 Files)
Volumetric water content θ (–)	`theta_files_dir`	HDF5 (`.h5`), one 2D file per time step and depth layer	`Strength_variation_with_theta = 2` (Section 6, HDF5 Files)
Peak ground acceleration PGA (g)	`pga_file`	HDF5 (`.h5`), 2D	Coupled rainfall + earthquake mode (`Triggering_Indicator = 2`)
Peak ground velocity PGV (cm/s)	`pgv_file`	HDF5 (`.h5`), 2D	Coupled mode with the Bray & Macedo model (`Seismic_displacement_model = 4`) or a Rathje model with the PGAV method (`Seismic_displacement_model = 2` or `3` , `Rathje_method = 2`)

Input	Parameter (key in `params`)	File format	Required when
Effective cohesion c' (kPa), every cell and depth layer	`cohesion_3d_file`	HDF5 (`.h5`), 3D	`Strength_indicator = 2` (Section 5, 3D Shear Strength Files)
Effective friction angle φ' (degrees), every cell and depth layer	`phi_3d_file`	HDF5 (`.h5`), 3D	`Strength_indicator = 2` (Section 5, 3D Shear Strength Files)

Pore pressure heads and water contents can also come from a coupled Landlab component instead of files; this, and the constant R_u alternative, are described together in Section 6. All raster files are north-up, like the DEM (Section 5, Raster Orientation).

DEM (Esri ASCII Grid)

The DEM is an Esri ASCII grid (`.asc`), given by the `dem_file` parameter. You load it onto the Landlab grid with Landlab's `read_esri_ascii` , and CRISIS reads its header to compute the coordinates of every cell; no separate coordinate files are needed. The cell size (`cellsize`) is also read automatically from the header. The header looks like this:

```
ncols         100
nrows         80
xllcorner     500000.0      (or xllcenter)
yllcorner     4000000.0     (or yllcenter)
cellsize      1.0
NODATA_value  -9999
```

> **Note:** The DEM header `cellsize` and coordinates must be in meters, because landslide area and volume are computed directly from `cellsize` (the area of one cell is `cellsize`²) and the coordinates are used to place the output polygons. For the same reason, `Coordinate_Reference_System` must be a **projected** CRS (e.g. any UTM zone, State Plane, Albers), not a **geographic** one (e.g. WGS 84, NAD 83). A geographic CRS locates points as longitude/latitude angles in degrees; selecting one would tag the output shapefiles with degree-based units while their coordinates remain meter-scale numbers, making them unusable in GIS software. `nrows` and `ncols` must equal the shape of the Landlab grid, and `cellsize` must equal `grid.dx` and `grid.dy`; CRISIS checks this and raises a `ValueError` otherwise. Cells with an elevation below −900 (e.g. `NODATA_value -9999`) or NaN are treated as no-data and never fail.

Seismic Rasters (Coupled mode only, Triggering_Indicator = 2)

These files are **required only when** `Triggering_Indicator = 2` (coupled rainfall + earthquake mode). When `Triggering_Indicator = 1` (rainfall only), they are not read.

File	Description
`PGA.h5`	Peak ground acceleration (g)
`PGV.h5`	Peak ground velocity (cm/s), **required only for the Bray & Macedo model or the Rathje PGAV method**

3D Shear Strength Files (Strength_indicator = 2)

Shear strength is controlled by `Strength_indicator`: set to `1` for spatially uniform effective cohesion c' and effective friction angle φ' applied to all cells and depths (scalar values, no files needed), or set to `2` to load spatially varying 3D arrays of c' and φ'. The files below are **required only when** `Strength_indicator = 2`.

File	Description
`cohesion_3d.h5`	Effective cohesion c' (kPa), shape `(nrows, ncols, Depth_points)`
`phi_3d.h5`	Effective friction angle φ' (degrees), shape `(nrows, ncols, Depth_points)`

Note: these are the requirements for `cohesion_3d.h5` and `phi_3d.h5`:

Shape: `(nrows, ncols, Depth_points)`, in that order. `nrows` and `ncols` must equal the DEM header values.

- **Axis 0 (rows):** north to south (north-up). Row 0 is the northern edge, the same order as the data lines in `dem.asc` (Section 5, Raster Orientation).
- **Axis 1 (columns):** west to east. Column 0 is the western edge.
- **Axis 2 (depth layers):** shallow to deep.

HDF5 File Format

All input files other than the DEM are HDF5 files:

- **Format:** HDF5, with the extension `.h5`.
- **Content:** exactly one array per file, stored under the dataset name `data` (the label used to find the array inside the file). The name of the variable in your NumPy or MATLAB code does not matter.
- **Shape:** 2D `(nrows, ncols)` for PGA, PGV, and each pore pressure head or water content file; 3D `(nrows, ncols, Depth_points)` for the cohesion and friction angle arrays.
- **Size:** the same number of rows and columns as the DEM.
- **Orientation:** north-up, like the DEM (Section 5, Raster Orientation).

Why HDF5? HDF5 is a binary storage format (with optional compression), which makes it far more efficient than plain-text raster formats: files are smaller on disk, and reading/writing them is substantially faster, especially for the large 3D arrays (rows × cols × depth layers) used for cohesion, friction angle, pore pressure, and volumetric water content across many time steps.

If your rasters are currently saved as **ASCII files**, read them into Python or MATLAB as a raster matrix, **without the header**, and then write that matrix into an HDF5 file using the Write examples below.

Python, Read ASCII raster (header skipped)

```
import numpy as np

# skip the 6-line Esri ASCII header (skiprows=5 if it has no NODATA_value line)
matrix = np.loadtxt('File.asc', skiprows=6)
```

Python, Write (2D or 3D raster matrix → HDF5)

```
import numpy as np
import h5py

# matrix is your 2D (rows × cols) or 3D (rows × cols × depth) NumPy array
with h5py.File('File.h5', 'w') as f:
    f.create_dataset('data', data=matrix)   # 'data' is the dataset key inside the file
```

Python, Read

```
import h5py

with h5py.File('File.h5', 'r') as f:
    matrix = f['data'][:]   # reads the dataset stored under the key 'data'
```

MATLAB, Read ASCII raster (header skipped)

```
% arcgridread() (Mapping Toolbox) reads an Esri ASCII grid and returns the matrix without its header
matrix = arcgridread('File.asc');
```

MATLAB, Write (2D or 3D raster matrix → HDF5)

```
filename = 'File.h5';
% matrix is your 2D (rows × cols) or 3D (rows × cols × depth) MATLAB array
% MATLAB stores arrays column-major: reverse the dimensions so that Python
% (and CRISIS) read the file as rows × cols (× depth)
out = permute(matrix, ndims(matrix):-1:1);
h5create(filename, '/data', size(out));   % '/data' is the dataset key inside the file
h5write(filename, '/data', out);
```

MATLAB, Read

```
m = h5read('File.h5', '/data');          % read the dataset stored under the key '/data'
matrix = permute(m, ndims(m):-1:1);      % back to rows × cols (× depth)
```

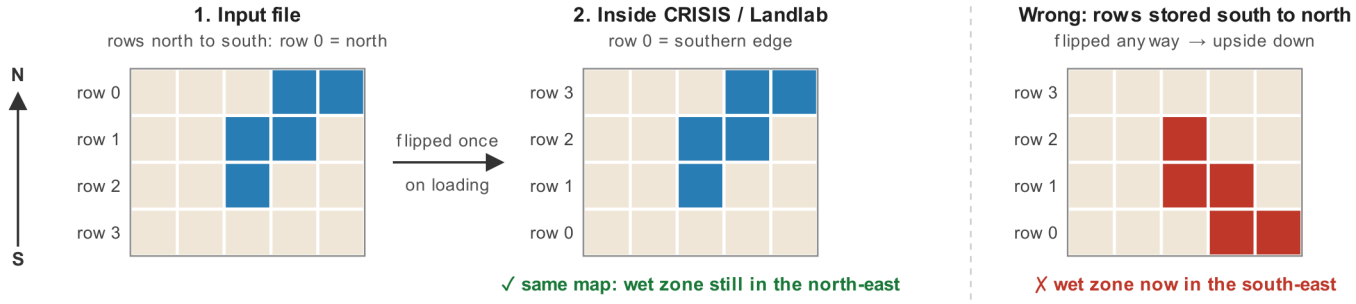
Note: MATLAB reverses the order of the dimensions of HDF5 arrays (it stores arrays column-major). Without the `permute` lines above, a file written by MATLAB is read transposed (2D: cols × rows; 3D: depth × cols × rows) and either fails with a shape error or is applied to the wrong cells. To check a file, read it in Python: `f['data'].shape` must be `(nrows, ncols)` or `(nrows, ncols, Depth_points)`.

Raster Orientation

A raster is stored as a table of rows and columns, so every raster needs a convention for which edge of the map is row 0. CRISIS uses one rule for all of them:

- **Input files: north-up.** Row 0 is the northern edge (the largest y coordinate), each following row lies one cell further south, and columns run from west to east. This is how Esri ASCII grids are written and how ArcGIS, QGIS, and MATLAB normally store rasters. It applies to the DEM (`.asc`) and to every HDF5 file: pore pressure heads, water contents, PGA, PGV, cohesion, and friction angle.
- **Inside the CRISIS Landlab component: south-up.** Row 0 is the southern edge, as Landlab requires (Landlab numbers its grid nodes from the lower-left corner). **The component does this conversion itself, not the user:** every input file is flipped exactly once, when it is loaded (the DEM by Landlab's `read_esri_ascii` , the HDF5 files by the core's `load_h5` function).
- **Output files: north-up.** The component converts the results back: the triggering-cell indices of the Excel workbooks count rows from the top (north) of the DEM file, and the shapefiles are placed by their real-world coordinates (Section 8).
- **Grid fields: south-up.** Everything stored on the Landlab grid stays in Landlab's order: the output grid fields (Section 8), the 2D arrays `landslide_fid` and `failed_cells` (Section 9), and the pore pressure and water content fields of indicator 3, which a coupled Landlab component writes already south-up and CRISIS does not flip (Section 6, Grid Fields). To plot one of them with north at the top, use Landlab's `imshow_grid` , or flip the 2D array with `np.flipud` .

So the user only has to make sure that every input file is north-up, and can read every output file as north-up. The flip does not change the map; it only renumbers the rows, so each cell keeps its position on the ground:



The same 4 × 5 map as stored in an input file (1) and inside CRISIS after loading (2). If a file stores its rows the other way round, from south to north (right), the flip turns it upside down and the values are applied to the wrong cells.

Important: make every input file north-up, and do not flip it in any other way. CRISIS does all the flipping itself:

- **DEM (`.asc`):** Esri ASCII grids are always north-up, so the DEM is used as it is.
- **Pore pressure head or water content files (`.h5`) from an external hydrological model:** make sure their rows are ordered from north to south before saving them. Some models (for example ParFlow) number their grid from the south; their output must be flipped once (to rows from north to south) before it is saved for CRISIS, otherwise the pore pressure heads or water contents are applied to the mirror-image cells.
- **All other `.h5` files (PGA, PGV, cohesion, friction angle):** also north-up.

A quick check: plot a file with its first row at the top. It should look like a map of the area, matching the DEM.

---

6. Pore Pressures and Water Contents

`Hydrological_model_indicator` sets where the pore pressure heads ψ come from, and `Strength_variation_with_theta` where the volumetric water contents θ come from; the water contents set Bishop's effective stress coefficient χ. Each indicator takes one of three values:

Indicator	`Hydrological_model_indicator` (pore pressure heads)	`Strength_variation_with_theta` (water contents)
`1`	Steady-state, spatially uniform pore pressure coefficient R_u (Section 6, Constant R_u)	None: fully saturated, χ = 1
`2`	HDF5 files `Pressure_head_T{t}Z{z}.h5`, e.g. from ParFlow (Section 6, HDF5 Files)	HDF5 files `Theta_T{t}Z{z}.h5` (Section 6, HDF5 Files)
`3`	Grid field `soil_water__pressure_head`, written by a coupled Landlab component (Section 6, Grid Fields)	Grid field `soil_water__volume_fraction`, written by a coupled Landlab component (Section 6, Grid Fields)

Indicator 2 or 3: what is the difference? Both give pore pressure heads (or water contents) that vary in space and time. They differ in how the hydrological model is run and how its results reach CRISIS:

- **Indicator 2, external hydrological model:** the hydrological model (e.g. ParFlow) is run on its own, outside Landlab, for all time steps, before CRISIS. Its results are saved as HDF5 files, one per time step and depth layer, and CRISIS reads the files of each time step in turn (and copies them into the grid fields, so other components can use them).
- **Indicator 3, coupled Landlab component:** a hydrological Landlab component runs together with CRISIS, in the same Landlab time loop. At every time step it computes the pore pressure heads (or water contents) and writes them to the grid fields; CRISIS then reads them for that time step. No files are needed, and the two components work together, one time step at a time.

The table below lists every combination, with where the pore pressure heads ψ and Bishop's coefficient χ (Section 6, Bishop's Effective Stress Coefficient χ) come from:

Hydrological model indicator	Strength variation with theta indicator	Pore pressure heads ψ	Bishop's coefficient χ
`1`	`1`, `2` or `3` (ignored)	Constant R_u: $\psi(z) = R_u\,\gamma\,z/\gamma_w$ (Eq. 3), steady state	$\chi = 1$
`2`	`1`	HDF5 files `Pressure_head_T{t}Z{z}.h5`	$\chi = 1$
`2`	`2`	HDF5 files `Pressure_head_T{t}Z{z}.h5`	Eq. 4, θ from the HDF5 files `Theta_T{t}Z{z}.h5`
`2`	`3`	HDF5 files `Pressure_head_T{t}Z{z}.h5`	Eq. 4, θ from the grid field `soil_water__volume_fraction`
`3`	`1`	Grid field `soil_water__pressure_head`	$\chi = 1$
`3`	`2`	Grid field `soil_water__pressure_head`	Eq. 4, θ from the HDF5 files `Theta_T{t}Z{z}.h5`
`3`	`3`	Grid field `soil_water__pressure_head`	Eq. 4, θ from the grid field `soil_water__volume_fraction`

With `Hydrological_model_indicator = 2` or `3`, the pore pressure head of each layer is capped at the layer depth ($\psi \le z$), so the water table can never rise above the ground surface. With `Hydrological_model_indicator = 1`, `Strength_variation_with_theta` is ignored (any of `1`, `2` or `3`; CRISIS prints a notice if it is not `1`), and no water content is needed.

> **Note:** A "time step" has no fixed real-world duration in CRISIS itself: it can represent an hour, 30 minutes, a day, or any other interval. That duration does not enter the slope stability calculations at all; it is defined entirely by the hydrological model (HDF5 files, or the Landlab component writing the grid fields) that produced your pore pressure heads and water contents. CRISIS simply processes them in index order, `t = 0, 1, 2, …`. Keep track of that mapping yourself (e.g. "each time step = 1 hour") to correctly interpret outputs indexed by time step, `Earthquake_time_point`, the rows of `Total_landslides_with_time.xlsx`, and the per-time-step columns of `Individual_landslides_with_time.xlsx` (both labeled `t=0`, `t=1`, …).

Number of time steps. The number of time steps is never a parameter: with a constant R_u (`Hydrological_model_indicator = 1`) the run has a single time step; with HDF5 files (indicator 2) CRISIS counts the time steps of the files when the component is created, and every time step found is run; with grid fields written by a coupled Landlab component (indicator 3) there is no fixed number, and the run has as many time steps as `run_one_step()` is called in your time loop. The property `crisis.number_of_time_steps` gives the number (`None` with indicator 3), e.g. `for t in range(crisis.number_of_time_steps): crisis.run_one_step()`.

Constant R_u (Hydrological_model_indicator = 1)

The pore pressure is given by a steady-state, spatially uniform pore pressure coefficient R_u (parameter `Ru`), the same at every cell and constant in time; no pore pressure files are needed. The pore pressure coefficient is defined as (Bishop & Morgenstern, 1960):

$$R_u = \frac{u}{\gamma\, z} = \frac{\psi\, \gamma_w}{\gamma\, z} \qquad (1)$$

where u is the pore water pressure (kPa); γ and γ_w are the respective soil and water unit weights; z is the vertical depth (m); and ψ is the pore pressure head (m).

Note: R_u does not range from 0 to 1; it ranges from 0 (no pore pressure, ψ = 0) up to $R_{u,max}$, which corresponds to fully saturated conditions, i.e. pressure head equal to depth ($\psi = z$). Substituting $\psi = z$ into the definition above (Eq. 1) gives:

$$R_{u,max} = \frac{\gamma_w}{\gamma} \qquad (2)$$

Since γ (soil) is always greater than γ_w (water, 9.81 kN/m³), $R_{u,max}$ is always less than 1, e.g. for γ = 22 kN/m³, $R_{u,max}$ ≈ 0.446. Entering an `Ru` value above $R_{u,max}$ for your soil's unit weight corresponds to a pore pressure head greater than the depth itself, which is not physically achievable. CRISIS does not check or cap `Ru`: a value above $R_{u,max}$ is used as given.

Note: Even with a constant R_u (no pore pressure files), you still need to discretize the subsurface via `z_min`, `z_max`, and `Depth_points`. From the definition above (Eq. 1), the pressure head at depth z works out to:

$$\psi(z) = \frac{R_u \gamma z}{\gamma_w} \qquad (3)$$

so R_u acts as a constant slope describing how pressure head grows with depth. Define `z_min` / `z_max` to span the subsurface thickness you want to search, then set `Depth_points` high enough to resolve that range into distinct candidate failure surfaces.

Because $\psi(z)$ above depends only on z, every cell at a given depth has exactly the same pressure head at every location and every time step: a constant R_u has no spatial variability and does not change in time. It is a steady-state condition: CRISIS writes steady-state outputs without time steps (Section 8), and CRISIS runs it as a single time step automatically. Only `Hydrological_model_indicator = 2` (HDF5 files) and `3` (a grid field) capture pressure heads that vary in space and time. CRISIS also writes this pressure head into the grid field `soil_water__pressure_head`, once, when the component is created.

HDF5 Files (Hydrological_model_indicator = 2 or Strength_variation_with_theta = 2)

Pore pressure head files (`Hydrological_model_indicator = 2`)

One file per time step and depth layer, named exactly: `Pressure_head_T{t}Z{z}.h5`, where `t` is the time index and `z` the depth index, each holding a 2D raster of pore pressure head (meters).

Example: `Pressure_head_T3Z2.h5` = time step 3, depth layer 2, i.e. the second layer counted **from the ground surface down**. The time index T is 0-based: `T0`, `T1`, … without gaps. CRISIS counts the time steps from these files and runs every time step found, so the folder must hold only the time steps of this run; every time step must have exactly the depth layers `Z1` … `Z{Depth_points}`. A gap in the time steps, a time step missing one of its depth layers, a time step with depth layers beyond `Depth_points`, or water content files that do not cover the same time steps as the pore pressure head files stop the run when the component is created, with a message naming the problem. The depth index Z is 1-based and runs from `1` to `Depth_points`, counted downward from the ground surface: Z1 is the shallowest layer, at depth `z_min` below the ground surface, and each following layer lies deeper, down to the last layer at depth `z_max`. The layers are evenly spaced, so layer Z_k lies at the depth

$$z_k = z_{min} + (k-1)\,\frac{z_{max} - z_{min}}{\texttt{Depth_points} - 1}$$

below the ground surface, measured vertically. For example, with `z_min = 0.125` m, `z_max = 1.875` m and `Depth_points = 8` (the Example notebook), the layers lie every 0.25 m, and Z2 is at 0.375 m below the ground surface. Every layer, including Z1, is checked for failure, unless it is at depth 0, the ground surface (see the note on `z_min` in Section 7). A file must exist for every combination of T and Z.

The files are read from the folder `hydro_files_dir`. On loading, each file is flipped to Landlab's row order (Section 5, Raster Orientation), each pressure head is capped at the depth of its layer ($\psi \le z$), and the capped values of the time step are written into the grid field `soil_water__pressure_head`, so other components can read them.

These files are produced by an external transient groundwater flow model. Examples of commonly used models include ParFlow, TRIGRS, MODFLOW, etc.

Water content files (`Strength_variation_with_theta = 2`)

One file per time step and depth layer, named exactly: `Theta_T{t}Z{z}.h5`, each holding a 2D raster of volumetric water content (unitless, 0–1), with the same T/Z indexing as the pore pressure files. The files are read from the folder `theta_files_dir`, flipped to Landlab's row order, and written into the grid field `soil_water__volume_fraction`. Like the pore pressure head files, they are produced by the external transient groundwater flow model (e.g. ParFlow, TRIGRS, MODFLOW, etc.); they are used to compute Bishop's coefficient χ (Section 6, Bishop's Effective Stress Coefficient χ).

Grid Fields from a Coupled Landlab Component (Hydrological_model_indicator = 3 or Strength_variation_with_theta = 3)

With `Hydrological_model_indicator = 3` (and/or `Strength_variation_with_theta = 3`), the pore pressure heads (and/or water contents) are not read from files: a coupled Landlab component, for example a groundwater flow component run before CRISIS in the same time loop, writes them on the grid at each time step, and CRISIS reads them from these node fields:

- `soil_water__pressure_head`: pore pressure heads (meters);
- `soil_water__volume_fraction`: volumetric water contents (unitless, 0–1).

The fields must satisfy:

- **Shape** `(number_of_nodes, Depth_points)`: one row per node, in Landlab's order (row 0 = southern edge of the grid), one column per depth layer, from `z_min` to `z_max`.
- **Indicator = 3:** the field must be on the grid before `CrisisComponent` is created, and the other component must update it before each `run_one_step()`. CRISIS reads it and does not modify it.
- **Indicator = 1 or 2:** CRISIS writes the values it uses into the field, so other components can read them: `soil_water__pressure_head` with `Hydrological_model_indicator = 1` (the R_u heads, filled once, when the component is created) or `2` (the capped heads of the current time step), and `soil_water__volume_fraction` with `Strength_variation_with_theta = 2`. No water content field is created when `Strength_variation_with_theta = 1`.
- Any floating-point type (e.g. float64, float32); integer fields are rejected. Other field names can be given with the `pressure_head` and `water_content` arguments of `CrisisComponent` (Section 9).

Bishop's Effective Stress Coefficient χ

The effective stress formulation is controlled by `Strength_variation_with_theta`: set to `1` to assume full saturation everywhere (Bishop's χ = 1, no water content needed), or set to `2` (HDF5 files) or `3` (grid field) to compute χ from spatially and temporally varying volumetric water contents. χ scales

the pore pressure contribution based on the degree of saturation at each grid cell, depth, and time step:

$$\chi = \frac{\theta(x, y, z, t) - \theta_r}{\theta_s - \theta_r} \qquad (4)$$

where $\theta(x, y, z, t)$ is the volumetric water content at a given grid cell, depth, and time instant, computed by the groundwater flow model; θ_s is the saturated volumetric water content (`Theta_s`); and θ_r is the residual volumetric water content (`Theta_r`). The result is clipped to the range [0, 1].

---

7. Parameters (the `params` Dictionary)

All parameters are the keys of a single Python dictionary, `params` . The user writes this dictionary in their own Python script (or notebook), the same script in which they call the CRISIS Landlab component; there is no separate input file. The dictionary is passed to the component when it is created:

```
params = {"Gamma": 22.0, "Strength_indicator": 1, "c": 20.0, "Phi": 37.0, ...}   # all keys: tables below
crisis = CrisisComponent(grid, params)
```

A complete script is given in Section 9. The component reads these values, opens the files they point to, and passes the resulting arrays and numbers to the functions of the CRISIS core; the core itself never reads `params` .

No parameter has a default value. Every key the selected settings need must be given; keys the settings do not use can be omitted (or set to `None` / `""`). Before anything else, the component checks `params` (function `check_params`): if a needed key is missing, or a choice key has a value that is not allowed, it stops with a `ValueError` that lists every such key and the setting that needs it, for example:

```
ValueError: Missing or invalid entries in params:
  - Ru is missing (required when Hydrological_model_indicator = 1)
  - cohesion_3d_file, phi_3d_file are missing (required when Strength_indicator = 2)
```

The keys needed are:

Setting	Required keys
Every run	`dem_file`, `output_dir`, `Coordinate_Reference_System`, `Gamma`, `z_min`, `z_max`, `Depth_points`, `Min_LD_area`, `top_or_bottom_indicator`, `Triggering_Indicator`, `Hydrological_model_indicator`, `Strength_indicator`
`Hydrological_model_indicator = 1`	`Ru` (a single time step, so `Earthquake_time_point` must be 0)
`Hydrological_model_indicator = 2`	`hydro_files_dir`, `Strength_variation_with_theta`
`Hydrological_model_indicator = 3`	`Strength_variation_with_theta`
`Strength_variation_with_theta = 2`	`theta_files_dir`, `Theta_s`, `Theta_r`
`Strength_variation_with_theta = 3`	`Theta_s`, `Theta_r`
`Strength_indicator = 1`	`c`, `Phi`
`Strength_indicator = 2`	`cohesion_3d_file`, `phi_3d_file`
`Triggering_Indicator = 2`	`pga_file`, `Earthquake_time_point`, `Seismic_displacement_threshold`, `Seismic_displacement_model`, and the keys of the model below
Model 1 (Jibson)	`M`
Model 2 (Rathje rigid)	`Rathje_method`; then `M` (PGAM, `Rathje_method = 1`) or `pgv_file` (PGAV, `Rathje_method = 2`)
Model 3 (Rathje flexible)	`Rathje_method`, `M`, `Ts`, the fault corners `fault_X1` … `fault_Z4`; and `pgv_file` with `Rathje_method = 2`
Model 4 (Bray & Macedo)	`M`, `Ts`, `pgv_file`, the fault corners `fault_X1` … `fault_Z4`

Files and Folders

Slope, flow direction, and drainage area are computed by Landlab from the DEM, and the cell size is read from the DEM header (Section 5, DEM).

Parameter	Description
`dem_file`	Esri ASCII grid (`.asc`) of the DEM; its header gives the cell coordinates and the cell size (`cellsize` , in meters), which must match the Landlab grid spacing (Section 5, DEM)
`output_dir`	Folder the output files (shapefiles and Excel workbooks) are written into; created automatically if it does not exist
`Coordinate_Reference_System`	EPSG code of the projected CRS used for the output shapefiles (e.g. `EPSG:6566`)

Triggering Event

Parameter	Description
`Triggering_Indicator`	1 = rainfall only · 2 = coupled rainfall + earthquake

Earthquake Inputs (Coupled mode only, `Triggering_Indicator = 2`)

Parameter	Description
`Seismic_displacement_model`	1 = Jibson (2007) · 2 = Rathje et al. (2014) Rigid · 3 = Rathje & Antonakos (2011) Flexible · 4 = Bray & Macedo (2019)
`Rathje_method`	1 = PGAM · 2 = PGAV **(Rathje models 2 and 3 only)**
`pga_file`	HDF5 file, PGA raster (g)
`pgv_file`	HDF5 file, PGV raster (cm/s) **(Bray & Macedo model and Rathje PGAV method only)**

Parameter	Description
`Earthquake_time_point`	Time step index (0-based) at which the earthquake occurs, an integer between 0 and the number of time steps − 1 (0 with a constant R_u; Section 6). Seismic displacement is evaluated at this time step only. With a coupled component (`Hydrological_model_indicator = 3`) the number of time steps is not known in advance: `write_outputs()` prints a notice if this time step was never reached
`Seismic_displacement_threshold`	Newmark displacement threshold for seismic failure (cm), typically 5 cm
`M`	Earthquake moment magnitude M_w
`Ts`	Fundamental period of the sliding mass (s). Used by the Rathje & Antonakos (2011) and Bray & Macedo (2019) models **(models 3 and 4 only)**

The keys each displacement model needs are listed in the table at the top of this section. With a constant R_u (a single time step), set `Earthquake_time_point = 0`.

Required only for the Rathje & Antonakos (2011) Flexible and Bray & Macedo (2019) displacement models (no default corners), the fault rupture plane is defined by four corner coordinates (X and Y in meters in the same projected CRS as the DEM; Z in meters depth, positive downward, so the surface is Z = 0 and a 15 km deep corner is Z = 15 000 m). Corners must follow a consistent winding order: top-left → top-right → bottom-right → bottom-left (top = shallow edge, bottom = deep edge). The closest distance from every cell to this plane is computed once at the start of the run.

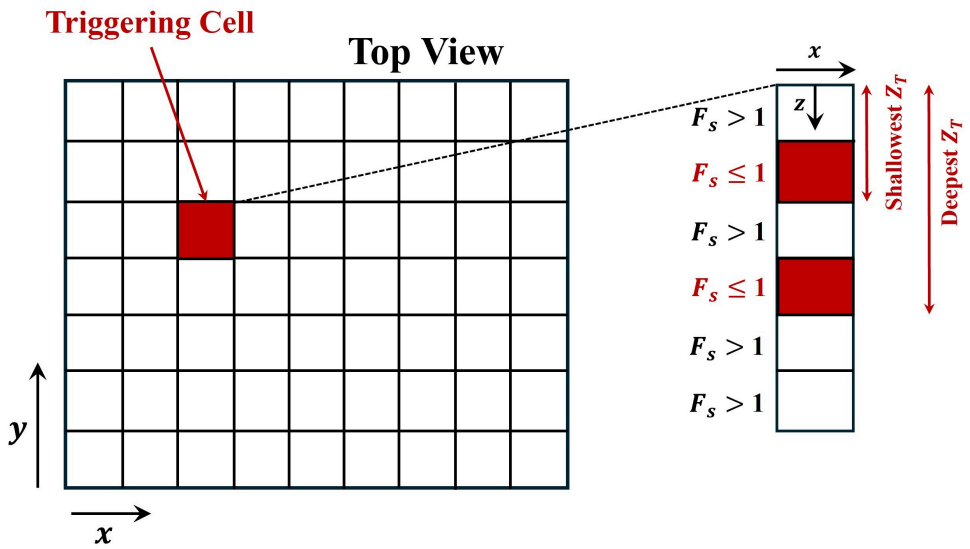
Parameter	Description
`fault_X1` , `fault_Y1` , `fault_Z1`	Easting, northing, and depth of the top-left fault corner (meters)
`fault_X2` , `fault_Y2` , `fault_Z2`	Easting, northing, and depth of the top-right fault corner (meters)

Parameter	Description
`fault_X3`, `fault_Y3`, `fault_Z3`	Easting, northing, and depth of the bottom-right fault corner (meters)
`fault_X4`, `fault_Y4`, `fault_Z4`	Easting, northing, and depth of the bottom-left fault corner (meters)

Soil Column

Parameter	Description
`Gamma`	Total unit weight of soil γ (kN/m³). The unit weight of water is fixed at γ_w = 9.81 kN/m³
`z_min`	Depth of the shallowest depth layer (meters)
`z_max`	Depth of the deepest depth layer (meters)
`Depth_points`	Number of subsurface depth layers, evenly spaced from `z_min` to `z_max`
`Min_LD_area`	Minimum landslide area (m²). A landslide with fewer than ⌈ `Min_LD_area` / `cellsize`² ⌉ cells is rejected, and its cells remain available to other landslides (Section 10)
`top_or_bottom_indicator`	When more than one depth at a cell fails, selects which one is treated as the landslide's rupture surface for the pseudo-3D geometry search: 1 = shallowest triggering depth (Z_T) · 2 = deepest triggering depth (Z_T)

Note: Failure is checked at every depth layer, including the first one (`z_min`). The only exception is a layer at depth 0, the ground surface itself: with `z_min = 0`, the first layer is not checked, and failure is checked at layers 2 to `Depth_points` (so `Depth_points` must then be at least 2).



At a triggering cell where more than one depth fails, `top_or_bottom_indicator` selects whether the reported triggering depth Z_T is the shallowest or the deepest failing depth. Shown here for rainfall-induced landslides, where a triggering cell is defined by $F_s \leq 1$. For earthquake-induced landslides, a triggering cell is instead defined by seismic displacement reaching the user-specified threshold.

Pore Pressures and Water Contents

How these parameters work together is explained in Section 6.

Parameter	Description
`Hydrological_model_indicator`	1 = constant coefficient R_u, uniform in space and time (steady state) · 2 = pore pressure heads from the HDF5 files `Pressure_head_T{t}Z{z}.h5` · 3 = pore pressure heads from the grid field `soil_water__pressure_head`, written by a coupled Landlab component (Section 6)
`Ru`	Pore pressure coefficient, from 0 (no pore pressure) up to $R_{u,max}$ (Section 6, Constant R_u) **(Hydrological_model_indicator = 1 only)**
`hydro_files_dir`	Folder containing pore pressure head (meters) files, named `Pressure_head_T{t}Z{z}.h5` (e.g. `Pressure_head_T0Z1.h5`) **(Hydrological_model_indicator = 2 only)**

Parameter	Description
`Strength_variation_with_theta`	1 = fully saturated assumption, χ = 1 everywhere · 2 = χ computed from the water contents of the HDF5 files `Theta_T{t}Z{z}.h5` · 3 = χ computed from the water contents of the grid field `soil_water__volume_fraction`, written by a coupled Landlab component (Eq. 4; requires θ_s, θ_r; Section 6, Bishop's Effective Stress Coefficient χ). **(Hydrological_model_indicator = 2 or 3 only; with 1, not needed and ignored, χ = 1)**
`theta_files_dir`	Folder containing volumetric water content files, named `Theta_T{t}Z{z}.h5` (e.g. `Theta_T0Z1.h5`) **(Strength_variation_with_theta = 2 only)**
`Theta_s`	Saturated volumetric water content θ_s **(Strength_variation_with_theta = 2 or 3 only)**
`Theta_r`	Residual volumetric water content θ_r **(Strength_variation_with_theta = 2 or 3 only)**

Shear Strength

Parameter	Description
`Strength_indicator`	1 = spatially uniform: a single c' and φ' apply to all cells and depths · 2 = spatially variable 3D arrays: c' and φ' can differ at every cell and depth layer (Section 5, 3D Shear Strength Files)
`c`	Effective cohesion c' (kPa) **(Strength_indicator = 1 only)**
`Phi`	Effective friction angle φ' (degrees) **(Strength_indicator = 1 only)**
`cohesion_3d_file`	HDF5 file, 3D effective cohesion c' array of shape `(nrows, ncols, Depth_points)` (kPa) **(Strength_indicator = 2 only)**

Parameter	Description
`phi_3d_file`	HDF5 file, 3D effective friction angle φ' array of shape `(nrows, ncols, Depth_points)` (degrees) **(Strength_indicator = 2 only)**

---

8. Outputs

CRISIS produces two kinds of output: Landlab grid fields, updated at every time step and available to other Landlab components, and files, written to the folder `output_dir`.

Grid Fields

Per time step, reset at the start of every `run_one_step()`, and describing only the landslides that are new in that step:

Field	Type	Description
`landslide__failure_id`	integer	ID of the new landslide each cell belongs to, numbered 1…N within the current time step (restarting at 1 every step, unlike `landslide__failure_id_accumulated`); 0 elsewhere
`failure__depth`	float (m)	Depth of the failure surface at every cell of each new landslide; 0 elsewhere
`triggering__cells`	boolean	`True` at the triggering (initiation) cell of each new landslide

Cumulative over the run:

Field	Type	Description
`landslide__failure_id_accumulated`	float	ID of the landslide each failed cell belongs to; NaN where no failure has occurred
`landslide__failed_accumulated`	boolean	`True` if the cell has failed in any time step

Pore pressure heads and water contents of the current time step (one column per depth layer; Section 6, Grid Fields):

Field	Type	Description
`soil_water__pressure_head`	float (m)	Pore pressure heads; written by CRISIS with `Hydrological_model_indicator = 1` or `2`, read from a coupled component with `3`
`soil_water__volume_fraction`	float	Volumetric water contents; written by CRISIS with `Strength_variation_with_theta = 2`, read from a coupled component with `3`

Output Files

The shapefiles are written automatically after every time step, once at least one landslide has formed; the two Excel workbooks are written by `write_outputs()` (Section 9). The table lists the files of a run with pore pressure heads that change over time (`Hydrological_model_indicator = 2` or `3`); the files of a steady-state run with a constant R_u are described below the table.

File	Description
`Individual_landslides_with_time.xlsx`	10-sheet workbook, each column is a time step (header row `t=0`, `t=1`, …), each row is a landslide; empty cells mean no landslide. Five pairs of sheets, a "Current" and a "Cumulative" one for each quantity, with the units in the sheet name (`Triggering_Cells_Current` / `Triggering_Cells_Cumulative`, `Triggering_Depths_Current_m` / `Triggering_Depths_Cumulative_m`, `Triggering_Sources_Current` / `Triggering_Sources_Cumulative`, `Areas_Current_m2` / `Areas_Cumulative_m2`, `Volumes_Current_m3` / `Volumes_Cumulative_m3`): **Triggering_Cells** (flat linear index in Python 0-based row-major order, row × ncols + col, with rows counted from the top (north) of the ASCII DEM file); **Triggering_Depths** (depth in meters at which failure is initiated); **Triggering_Sources** (1 = rainfall only, 2 = rainfall + earthquake, 3 = earthquake only); **Areas** (landslide area in m²); **Volumes** (landslide volume in m³). "Current" sheets list only the landslides that are new at each time step; "Cumulative" sheets list every landslide up to that time step
`Total_landslides_with_time.xlsx`	One row per time step (column Time = `t=0`, `t=1`, …), current and cumulative number of landslides, total area (m^2), and total volume (m^3), with the units in the column names
`Predicted_landslides_time_{t}.shp`	Shapefile of all pseudo-3D landslide polygons up to time step `t`, one merged polygon per landslide, with its landslide ID in the `landslide_` attribute. Written only for time steps at which at least one landslide exists. The `.dbf`, `.prj`, `.shx`, and `.cpg` companion files must be kept in the same folder. Can be visualized in ArcGIS Pro or QGIS

Steady-state run (constant R_u, `Hydrological_model_indicator = 1`): the pore pressures do not change in time, so the run has a single time step and its output files have no time steps:

- `Predicted_landslides.shp`: a single shapefile of all landslide polygons (same content as above).

- `Individual_landslides.xlsx`: one sheet per quantity (Triggering_Cells, Triggering_Depths_m, Triggering_Sources, Areas_m2, Volumes_m3), each with a single column listing the landslides.
- `Total_landslides.xlsx`: a single row with the number of landslides, the total area (m^2), and the total volume (m^3).

Using the Results in Other Landlab Components

The per-step grid fields can be passed directly to other Landlab components. For example, to route the new landslides with `MassWastingRunout`:

```
grid.add_field("soil__thickness", grid.at_node["failure__depth"].copy(), at="node", clobber=True)
grid.add_field("mass__wasting_id", grid.at_node["landslide__failure_id"].copy(), at="node", clobber=True)
# then switch to a multiple-flow-direction router and run MassWastingRunout
```

---

9. Running CRISIS in Landlab

A CRISIS run is a short Python script (or notebook) that uses Landlab and the CRISIS Landlab component. The component is imported from Landlab, like any other Landlab component:

```
from landlab.components import CrisisComponent
```

The script then:

1. **Loads the DEM** onto a Landlab grid with `read_esri_ascii`.
2. **Computes slope, D8 flow direction, and drainage area** with Landlab (`calc_slope_at_node` and `FlowAccumulator`).
3. **Writes the `params` dictionary** (Section 7).
4. **Creates the component:** `crisis = CrisisComponent(grid, params)`.
5. **Runs the time steps:** calls `crisis.run_one_step()` once per time step: `crisis.number_of_time_steps` times with a constant R_u or HDF5 files, or once per step of your own time loop with a coupled component.
6. **Writes the Excel workbooks** (optional): `crisis.write_outputs()`.

Example

A complete run with a constant R_u and uniform shear strength:

```
import numpy as np
from landlab.io import read_esri_ascii
from landlab.components import CrisisComponent, FlowAccumulator

# 1. Grid from an Esri ASCII DEM
grid, dem = read_esri_ascii("Inputs/dem.asc", name="topographic__elevation")
grid.set_watershed_boundary_condition(node_data=dem, nodata_value=-9999.0)

# 2. Slope (degrees), D8 flow direction and drainage area with Landlab
FlowAccumulator(grid, flow_director="FlowDirectorD8").run_one_step()
grid.add_field("topographic__slope",
               np.degrees(grid.calc_slope_at_node("topographic__elevation")), at="node")

# 3. Parameters (Section 7)
params = {
    "output_dir": "Outputs",
    "Coordinate_Reference_System": "EPSG:6566",
    "dem_file": "Inputs/dem.asc",
    "Gamma": 22.0, "z_min": 0.0, "z_max": 2.0, "Depth_points": 9,
    "Min_LD_area": 12.0, "top_or_bottom_indicator": 1,
    "Triggering_Indicator": 1,                          # rainfall only
    "Hydrological_model_indicator": 1, "Ru": 0.42,      # constant Ru
    "Strength_variation_with_theta": 1, "Theta_s": 0.45, "Theta_r": 0.10,
    "Strength_indicator": 1, "c": 20.0, "Phi": 37.0,       # uniform strength
}

# 4.-6. Create the component, run the (single) time step, write the workbooks
crisis = CrisisComponent(grid, params)
crisis.run_one_step()          # a constant Ru has a single time step
crisis.write_outputs()
```

Creating the Component

```
CrisisComponent(
    grid,                                       # Landlab RasterModelGrid
    params,                                     # dictionary of parameters (Section 7)
    slope="topographic__slope",                 # grid field: slope in degrees
    drainage_area="drainage_area",              # grid field: drainage area in m²
    flow_direction="flow__receiver_node",       # grid field: D8 receiver node of each node
    log_callback=print,                         # receives progress messages
    pressure_head="soil_water__pressure_head",  # grid field: pore pressure heads (Section 6)
    water_content="soil_water__volume_fraction",  # grid field: water contents (Section 6)
)
```

The grid must already carry the elevation (`topographic__elevation`), slope, D8 flow direction, and drainage area fields (steps 1–2); if one is missing, the component stops with a `KeyError`. The field

names can be changed with the `slope`, `drainage_area`, and `flow_direction` arguments, and those of the pore pressure heads and water contents with the `pressure_head` and `water_content` arguments (see Section 6). When the component is created, it reads the DEM header and every input file that does not change with time, and prepares the outputs.

Methods and Properties

Member	Description
`run_one_step(dt=1.0)`	Runs one time step. The time step is counted internally (0, 1, 2, …) and selects the pore pressure heads and water contents of that step (HDF5 files or grid fields, Section 6); `dt` is accepted for compatibility with other Landlab components but is not used. Calling it more times than the run has time steps raises a `RuntimeError` (no limit with a coupled component, indicator 3).
`write_outputs()`	Writes the two Excel workbooks (Section 8) from the results accumulated so far.
`current_time_index`	Index of the next time step to run (starts at 0).
`number_of_time_steps`	Number of time steps of the run: 1 with a constant R_u, the number of time steps of the HDF5 files with indicator 2, `None` (no fixed number) with indicator 3 (Section 6).
`landslide_fid`	2D array of the landslide ID of every cell, over all time steps (NaN = no failure); south-up, like the grid fields.
`failed_cells`	2D boolean array: `True` where any failure has occurred; south-up, like the grid fields.
`state`	All results accumulated so far (counts, areas, volumes, triggering cells, depths, and sources, per time step).

Note: elevation, slope, and drainage area are re-read from the grid at every time step, so they can be updated by other Landlab components during a run; the flow direction is read once, when the component is created. CRISIS requires single-direction (D8) flow routing: if the grid is switched to a multiple-flow-direction router (e.g. for `MassWastingRunout`), do so after the CRISIS time steps.

Common Errors

Error	Cause
`TypeError`	`params` is not a dictionary, the grid is not a `RasterModelGrid`, or a pore pressure head or water content grid field does not hold floating-point values.
`KeyError`	The elevation, slope, flow direction, or drainage area field is missing, or (indicator 3) the pore pressure head or water content field is missing.
`ValueError`	A key the settings need is missing from `params`, or a choice key (indicator, model number, …) has a value that is not allowed: the message lists every such key (Section 7). Also: the DEM header does not match the grid, a layered grid field does not have the shape `(number_of_nodes, Depth_points)`, the water content files do not have the same time steps as the pore pressure head files, a time step of the HDF5 files has depth layers beyond `Depth_points`, `Earthquake_time_point` is not one of the time steps of the run, the DEM header misses `ncols`, `nrows` or `cellsize` (or gives both, or neither, of the lower-left corner and center), or the four fault corners lie on one line.
`FileNotFoundError`	A file given in `params` (`dem_file`, `pga_file`, `pgv_file`, `cohesion_3d_file`, `phi_3d_file`) does not exist; or, with HDF5 files (`Hydrological_model_indicator = 2` or `Strength_variation_with_theta = 2`), the folder holds no such files, a time step is missing (a gap), or a time step is missing one of its depth layers. All of this is found when the component is created. With a coupled component (indicator 3) and water content files, `run_one_step()` also stops if your loop runs past the last time step of the files.

Error	Cause
`RuntimeError`	`run_one_step()` was called more times than the run has time steps (`number_of_time_steps`).

Example Notebook

The `Example` folder contains a Jupyter notebook, `CRISIS_Example.ipynb`, that runs CRISIS on a small watershed in Utuado, Puerto Rico, under Hurricane Maria. The notebook runs CRISIS twice, with two of the three ways of supplying the pore pressure heads (Section 6):

1. **Spatially and temporally varying pore pressure heads** from an external hydrological model (ParFlow), one file per time step and depth layer (`Hydrological_model_indicator = 2`).
2. **A steady-state, spatially uniform pore pressure coefficient** R_u = 0.44 (`Hydrological_model_indicator = 1`), which needs no pore pressure files. Being steady state, this run has a single time step.

- **Inputs** (`Example/Input Files`): the DEM (`dem.asc`) and the pore pressure head files (`Pressure_head_T{t}Z{z}.h5` , used by run 1 only).

---

10. Model Physics

Rainfall-Induced Failure

The factor of safety F_s is computed at every raster cell, depth layer, and time step using the infinite slope stability equation (Duncan et al., 2014):

$$F_s = \frac{\tan\varphi'}{\tan\beta} + \frac{c' - \chi\,\psi\,\gamma_w \tan\varphi'}{\gamma\, z \sin\beta\cos\beta} \qquad (5)$$

where:

- c', effective cohesion (kPa)
- φ', effective friction angle (degrees)
- γ, unit weight of soil (kN/m³)
- γ_w, unit weight of water (9.81 kN/m³)

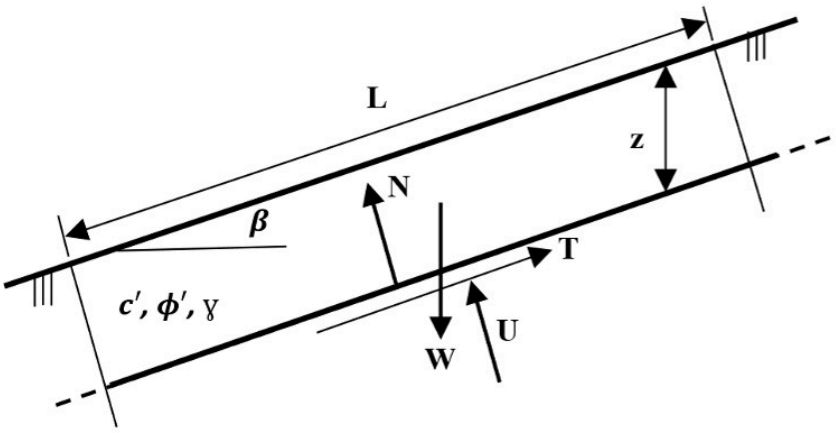
- z, vertical depth (m)
- β, slope angle (degrees)
- ψ, pore pressure head (m)
- χ, Bishop's effective stress coefficient (Eq. 4): $\chi = (\theta - \theta_r)/(\theta_s - \theta_r)$, clipped to the range [0, 1]. Set to 1 everywhere when `Strength_variation_with_theta = 1` or `Hydrological_model_indicator = 1` (full saturation assumed).

The pore pressure head ψ is obtained in one of two ways (Section 6):

- **Constant R_u** (indicator 1): ψ is derived from the user-supplied pore pressure coefficient (Eq. 3): $\psi(z) = R_u \gamma z/\gamma_w$
- **Variable pore pressure heads** (indicator 2 or 3): ψ varies spatially and over time: read from the HDF5 files `Pressure_head_T{t}Z{z}.h5` (indicator 2) or from the grid field `soil_water__pressure_head` written by a coupled Landlab component (indicator 3), and capped at the layer depth ($\psi \leq z$)

Where the effective normal stress becomes zero or negative (pore pressure uplift exceeding the overburden), the frictional term is dropped and only cohesion resists: $F_s = c'/(\gamma z \sin\beta \cos\beta)$.

A cell is classified as a **triggering cell** when $F_s \leq 1.0$ at any evaluated depth layer. Cells with a no-data elevation and near-flat cells (slope ≤ 0.0001°) are always treated as stable, and so is a depth layer at depth 0, the ground surface (see the note on `z_min` in Section 7).



Infinite slope stability limit equilibrium method (U is the pore water force, N is the normal force, W is the weight of the block, T is the resistive shear force acting at the base of the block, and L is the block length in translation).

Seismic Displacement (Coupled mode)

At the earthquake time step (`t = Earthquake_time_point`), each cell is checked for failure from rainfall, seismic displacement, or both; at all other time steps only rainfall-induced failure is evaluated. For seismic failure, the model computes the yield (critical) acceleration k_y (in g) at every cell and depth:

$$k_y = \frac{\dfrac{c'}{\gamma z \cos^2\beta} + \tan\varphi'\left(1 - \dfrac{\psi\,\gamma_w}{\gamma z \cos^2\beta}\right) - \tan\beta}{1 + \tan\varphi' \tan\beta} \qquad (6)$$

with a lower bound of 0.001 g. The Newmark displacement is then estimated using one of four empirical models, as implemented in the RESTAB model (Gong et al., 2023):

Model	Reference	Inputs used
1, Jibson	Jibson (2007)	PGA, M
2, Rathje Rigid	Rathje et al. (2014)	PGA, M (PGAM) or PGA, PGV (PGAV)
3, Rathje Flexible	Rathje & Antonakos (2011)	As model 2, plus T_s and rupture distance
4, Bray & Macedo	Bray & Macedo (2019)	M, T_s, and rupture distance (for the spectral acceleration at 1.3 T_s), plus PGV where PGV > 115 cm/s

Computed displacements are capped at 50 cm; with model 4, cells with k_y ≤ 0.01 g are assigned the 50 cm cap. A cell is classified as seismically unstable when the computed displacement reaches the user-specified `Seismic_displacement_threshold` (typically 5 cm). Models 3 and 4 require the fault plane corners (Section 7), from which the closest distance from every cell to the rupture is computed.

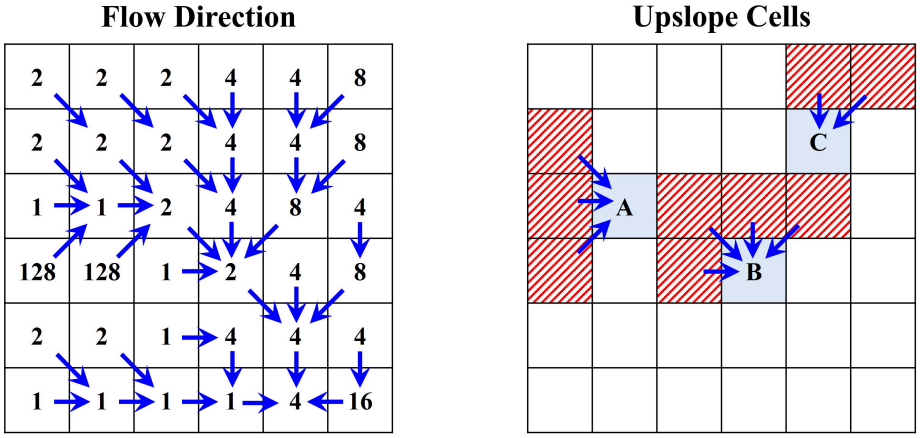
Each triggering cell records the source of its failure at the triggering depth: 1 = rainfall only, 2 = rainfall + earthquake, 3 = earthquake only.

Pseudo-3D Landslide Geometry

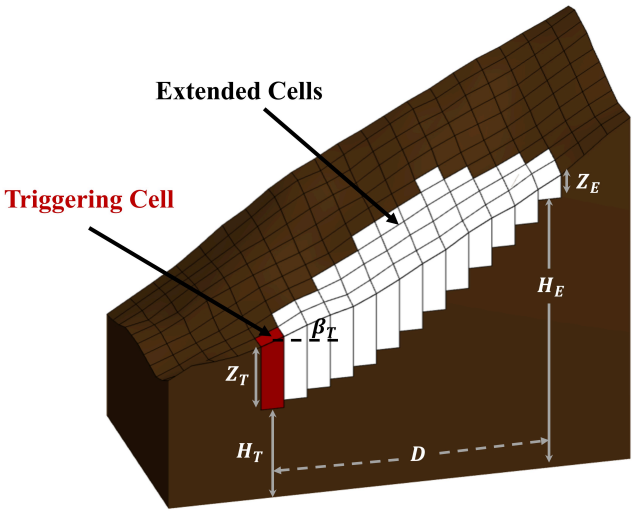
An upslope geomorphic projection algorithm expands each triggering cell, at its triggering depth, into a landslide. The algorithm uses an upslope search along the D8 flow network, constrained by the drainage (contributing) area. The result is a pseudo-3D landslide with a defined location, area, and volume that follows the natural drainage structure of the terrain.

Triggering cells are processed in descending order of upslope contributing area, so that failures with large contributing areas claim their upslope cells before smaller, nested ones. Triggering cells with the same contributing area are processed in the order of their Landlab node index (lowest first), so the results do not depend on the computer or software version. A triggering cell with no contributing area, or one that already belongs to a landslide, is skipped. Starting from each triggering cell, the search repeatedly adds the upslope neighbors that drain into a cell already in the landslide, provided that the neighbor:

1. does not already belong to a previously delineated landslide;
2. has an upslope contributing area no larger than the cell it drains into;
3. is not lower in elevation than the triggering cell; and
4. has a positive failure depth Z_E (Eq. 7).



Identification of upslope cells for a target cell from its eight neighboring cells using flow direction information.



Schematic of the pseudo-3D landslide formation methodology, starting at a triggering cell and expanding upslope to define the landslide geometry.

During the search, the depth of each extended cell is computed as:

$$Z_E = H_E - [D \cdot \tan(\beta_T) + (H_T - Z_T)] \qquad (7)$$

where H_E is the elevation of the extended cell; D is the horizontal Euclidean distance between the triggering and each extended cell; and β_T, H_T, and Z_T are the slope inclination, elevation, and triggering depth of the triggering cell, respectively (Gong et al., 2023).

In the pseudo-3D procedure, the failure surface initially has a constant slope inclination equal to the slope inclination at the triggering cell. Once the search is complete, the model adjusts the slope inclination at each extended cell, such that the slope of the failure plane becomes steeper as the distance from the triggering cell increases. This is achieved by first identifying the extended cell with the highest elevation and the minimum distance between this cell and the triggering cell (Min_dist). The model then computes the inclination angle M_{slope} (in degrees, rounded up to the next whole degree) of the line connecting this cell (at the surface) with the triggering cell at its triggering depth.

$$M_{slope} = \left\lceil \arctan\left(\frac{\max(\text{Elevation of all landslide cells}) - (H_T - Z_T)}{Min_dist} \right) \right\rceil \qquad (8)$$

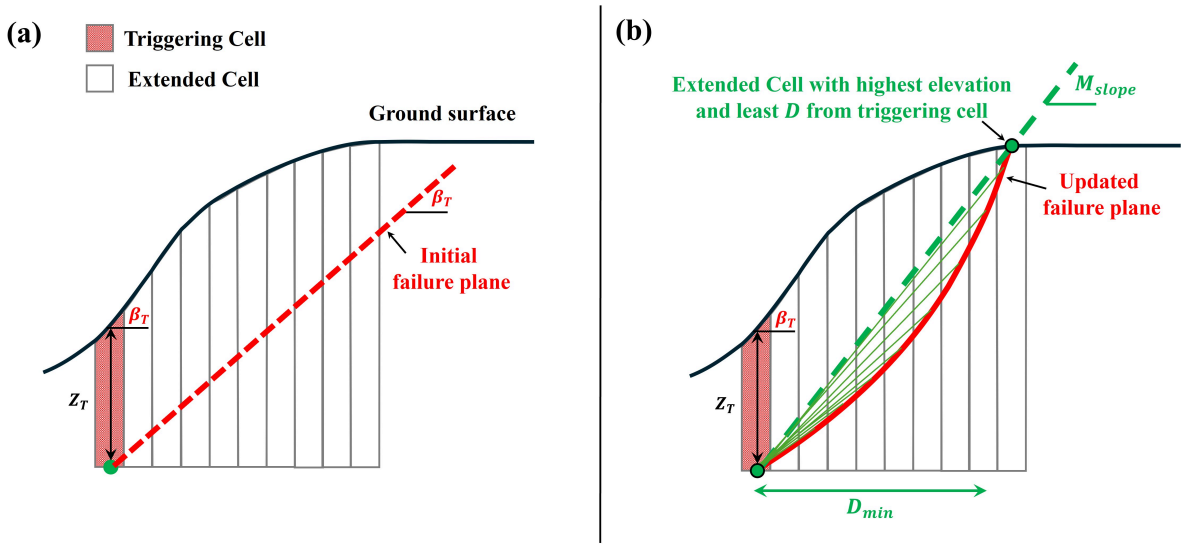
Subsequently, the rate of change in slope, dx_{slope}, is determined (and set to zero if negative):

$$dx_{slope} = \max\left(0,\ \frac{M_{slope} - \beta_T}{Min_dist}\right) \qquad (9)$$

The failure depth at every extended cell is then recomputed using this distance-dependent slope, replacing β_T in Eq. 7 with a linearly steepening angle:

$$H_F(D) = H_T - Z_T + D \cdot \tan\left(\beta_T + D \cdot dx_{slope}\right) \qquad (10)$$

where $H_F(D)$ is the elevation of the refined failure plane at distance D from the triggering cell. The final failure depth at each extended cell is $H_E - H_F(D)$, and only cells where this is still positive remain part of the landslide.



Schematic of the failure surface geometry: (a) initial surface with constant slope equal to that of the triggering cell; and (b) adjusted failure plane with slope increasing with distance from the triggering cell.

A landslide is accepted only if the upslope search yields more than two cells and, after this refinement, at least ⌈ `Min_LD_area` / `cellsize`^{2}⌉ cells remain. Its **area** is the number of cells × `cellsize`^{2}, and its **volume** is the sum of the final failure depths of its cells × `cellsize`^{2}. Once a cell belongs to a landslide, it cannot be part of another landslide in the same or any later time step.

Contact

For questions, issues, or feedback, please contact:

- Mirna Kassem, kassem_mirna@berkeley.edu
- Dimitrios Zekkos, zekkos@berkeley.edu

Acknowledgements

The authors thank Yuval Shmilovitz from the Community Surface Dynamics Modeling System (CSDMS) at the University of Colorado Boulder for help in creating this Landlab component.

This work was supported by the U.S. National Science Foundation (NSF) Division of Earth Sciences (EAR) through the Center for Land Surface Hazards (NSF award ID 2425607), UC Berkeley Engineering, the Jane Lewis Fellowship at UC Berkeley, and the SimCenter.

References

Bishop, A. W., & Morgenstern, N. (1960). Stability coefficients for earth slopes. *Géotechnique*, 10(4), 129–153. https://doi.org/10.1680/geot.1960.10.4.129

Bray, J. D., & Macedo, J. (2019). Procedure for estimating shear-induced seismic slope displacement for shallow crustal earthquakes. *Journal of Geotechnical and Geoenvironmental Engineering*, 145(12), 04019106. https://doi.org/10.1061/(ASCE)GT.1943-5606.0002143

Duncan, J. M., Wright, S. G., & Brandon, T. L. (2014). *Soil Strength and Slope Stability* (2nd ed.). John Wiley & Sons.

Gong, W., Zekkos, D., & Clark, M. (2023). A pseudo-3D methodology for regional-scale back-analysis of earthquake-induced landslides. *Engineering Geology*, 325, 107277. https://doi.org/10.1016/j.enggeo.2023.107277

Jibson, R. W. (2007). Regression models for estimating coseismic landslide displacement. *Engineering Geology*, 91(2-4), 209-218. https://doi.org/10.1016/j.enggeo.2007.01.013

Rathje, E. M., & Antonakos, G. (2011). A unified model for predicting earthquake-induced sliding displacements of rigid and flexible slopes. *Engineering Geology*, 122(1-2), 51-60. https://doi.org/10.1016/j.enggeo.2010.12.004

Rathje, E. M., Wang, Y., Stafford, P. J., Antonakos, G., & Saygili, G. (2014). Probabilistic assessment of the seismic performance of earth slopes. *Bulletin of Earthquake Engineering*, 12(3), 1071-1090. https://doi.org/10.1007/s10518-013-9485-9

Disclaimer & Liability

CRISIS is provided "as is," as open-source research software, without warranty of any kind, express or implied, including but not limited to warranties of merchantability, fitness for a particular purpose, and non-infringement. The authors make no representation or guarantee regarding the accuracy, completeness, or reliability of the model, its outputs, or any results derived from its use.

CRISIS is intended as a research and screening tool to support, not replace, professional engineering judgment, site-specific investigation, and analysis by a qualified geotechnical engineer or engineering geologist. It must not be used as the sole basis for operational decisions, hazard mitigation measures, regulatory compliance, or any decision affecting life safety or property.

In no event shall the authors or contributors be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the software or the use or other dealings in the software. Use of CRISIS, and reliance on any of its outputs, is entirely at the user's own risk.